



UNIVERSITATEA DIN
BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

ARHITECTURA PAN CA MODEL PENTRU APLICAȚII WEB FULL-STACK

Studiu de caz privind reducerea risipei alimentare

Absolvent: Eduard Emanuel Dinea

Coordonator științific: Conf. univ. dr. Marius Iulian Mihăilescu

București, iulie 2026

Rezumat

Această lucrare prezintă proiectarea și implementarea unei aplicații [web full-stack](#), denumite *Kookio*, realizată ca studiu de caz pentru analiza arhitecturii [PAN](#) în contextul aplicațiilor [web](#) moderne. Aplicația are ca obiectiv gestionarea inventarului de ingrediente pentru utilizatori casnici, contribuind indirect la reducerea risipei alimentare printr-o organizare mai eficientă a resurselor disponibile.

Motivația lucrării derivă din necesitatea adoptării unor arhitecturi [web](#) capabile să ofere performanță ridicată, coerență între [frontend](#) și [backend](#) și un proces de dezvoltare unitar. În acest cadru, lucrarea investighează arhitectura [PAN](#) ca model [full-stack](#) bazat pe ecosistemul [TypeScript](#), în care straturile aplicației sunt integrate într-un flux comun de dezvoltare.

Aplicația utilizează [SSR](#) cu [Angular](#) prin meta-framework-ul [Analog](#), comunicare *type-safe* între client și server prin [tRPC](#) și persistența datelor prin [Prisma](#) și [CockroachDB](#). Organizarea proiectului într-un [monorepo](#) gestionat cu [Nx](#) facilitează modularitatea, testarea automată și execuția incrementală a sarcinilor.

Implementarea este validată prin [testare unitară \(unit\)](#) și [E2E](#), folosind [Vitest](#) și [Playwright](#), precum și prin rularea aplicației într-un mediu modern de *deployment*. Rezultatele obținute indică faptul că arhitectura [PAN](#) constituie un model viabil pentru dezvoltarea aplicațiilor [web full-stack](#) scalabile, coerente și extensibile.

Cuprins

1	Introducere	1
	Motivație	1
	Scopul lucrării	1
	Contribuții ale lucrării	1
2	Preliminarii	3
2.1	Cadrul tehnologic	3
2.1.1	Arhitecturi web moderne	3
2.1.2	Angular și randare server-side	4
2.1.3	Monorepo și orchestrare cu Nx	4
2.1.4	tRPC și partajarea tipurilor	5
2.1.5	Persistență: Prisma și CockroachDB	5
2.1.6	Testare automată: Vitest și Playwright	5
2.1.7	Managementul configurației și mediilor	6
3	Implementare	7
3.1	Prezentare generală a implementării	7
3.1.1	Structura monorepo-ului Nx	7
3.1.2	Fluxul aplicației cap-coadă	8
3.2	Implementarea aplicației	8
3.2.1	Arhitecturi web moderne	8
3.2.2	Structura proiectului și organizarea codului	8
3.2.3	Angular și randare server-side	9
3.2.4	Gestionarea datelor cu Prisma și CockroachDB	9
3.2.5	Comunicarea client-server cu tRPC	10
3.2.6	Randare server-side și integrarea backend-ului	10
3.2.7	Jurnalizare și observabilitate	10

3.2.8	Fluxul de execuție al unei cereri	10
3.2.9	Testare și validare	11
3.2.10	Dezvoltare și deploy	12
4	Metodologie și evaluare	13
4.1	Setare experimentală	13
4.2	Indicatori și proceduri	13
4.2.1	Timp rulare CI (integrare continuă)	13
4.2.2	Latența DB (bază de date)	13
4.2.3	Fiabilitate E2E (testare cap-coadă)	13
4.3	Rezultate	13
4.4	Discuție și limitări	14
5	Concluzii	15
	Appendices	17
A	Anexă: Redactarea lucrării în L^AT_EX	17
A.1	Instrumente pentru redactare	17
A.2	Compatibilitate multiplatformă	17
B	Kookio: Manifest Arhitectural PAN (Prisma–Analog–Nx)	18
B.1	Viziune generală	18
B.2	Componentele arhitecturii	18
B.3	Structura fizică a proiectului	18
B.4	Avantaje comparative	19
B.5	Dir ecții de viitor	19
B.6	Concluzie	19
	Glosar de termeni	20
	Lista de acronime	21

Capitolul 1

Introducere

Motivație

În contextul vieții moderne, tot mai multe persoane caută soluții digitale care să simplifice organizarea meselor zilnice și gestionarea ingredientelor disponibile în gospodărie. Lipsa unei evidențe clare a ingredientelor conduce frecvent la achiziții redundante și, implicit, la risipă alimentară, fenomen cu impact atât economic, cât și social.

În paralel, aplicațiile [web](#) moderne se confruntă cu provocări tehnice semnificative. Printre acestea se numără performanța la încărcare, coerența dintre [frontend](#) și [backend](#), precum și mentenanța pe termen lung a codului. Aceste dificultăți devin mai accentuate în aplicațiile [full-stack](#), unde o integrare deficitară a straturilor conduce la creșterea complexității și a costurilor de dezvoltare.

Proiectul *Kookio* abordează aceste probleme prin adoptarea unei arhitecturi [web](#) moderne, bazate pe ecosistemul [TypeScript](#). Această arhitectură integrează [frontend](#)-ul, [backend](#)-ul și stratul de persistență într-un flux unitar de dezvoltare. În acest context, este analizată arhitectura [PAN](#) ca model pentru realizarea unor aplicații [web full-stack](#) coerente, extensibile și ușor de testat.

Scopul lucrării

Scopul acestei lucrări este proiectarea și implementarea unei aplicații [web full-stack](#) care permite gestionarea inventarului de ingrediente pentru utilizatori casnici. Aplicația servește drept studiu de caz pentru evaluarea arhitecturii [PAN](#) și pentru analiza impactului acesteia asupra dezvoltării aplicațiilor [web](#) moderne.

În acest sens, lucrarea își propune:

- realizarea unui sistem de gestiune a ingredientelor, cu persistență într-o bază de date relațională;
- integrarea [frontend](#)-ului și [backend](#)-ului într-un flux de dezvoltare unitar, cu partajarea tipurilor și a mecanismelor de validare;

- analiza beneficiilor arhitecturii **PAN** în ceea ce privește modularitatea, productivitatea și mentenanța aplicației;
- validarea implementării prin **testare unitară (unit)**, **testare de integrare (integration)** și **E2E**.

Contribuții ale lucrării

1. Un *blueprint* **PAN** complet funcțional: **Analog** (**Angular** 20) cu **SSR** peste **Nitro**, **tRPC** pentru **API** tipizat și **Prisma** către **CockroachDB**, într-un **monorepo Nx**.
2. Tipare de partajare a contractelor (**DTO-uri** și scheme **Zod** generate din **Prisma**) reutilizabile în **UI**, rute de server și teste.
3. Un pipeline **CI/CD** modular, bazat pe **workflow-uri** reutilizabile și *composite actions*, aliniat la țintele **Nx**; optimizări măsurabile ale timpului de rulare.
4. Metodologie de testare multi-nivel (**testare unitară (unit)**/**testare de integrare (integration)**/**E2E**) și practici de **observabilitate**/**jurnalizare (logging)** coerente în întregul ecosistem.

Aplicația este concepută ca un **MVP**, care poate fi extins ulterior cu funcționalități precum recomandarea de rețete sau integrarea cu furnizori externi. Prin aceste extensii, aplicația poate contribui indirect la reducerea risipei alimentare. Detaliile privind implementarea sunt prezentate în capitolul 3.

Capitolul 2

Preliminarii

Contextul tehnologic

Proiectul *Kookio* este fundamentat pe arhitectura unitară **PAN**, concepută pentru a acoperi întregul flux de dezvoltare **TypeScript**, de la interfața utilizatorului până la stratul de persistență. Această arhitectură consolidează într-un cadru comun mecanismele de orchestrare, **randare hibridă** și acces tipizat la date, oferind premisele unei aplicații coerente și extensibile [9, 6, 15].

Rolul acestui capitol este de a introduce conceptele și tehnologiile care stau la baza implementării, fără a intra încă în detalii de ordin practic. Manifestul arhitectural complet, alături de avantajele comparative și direcțiile de evoluție, este prezentat în **Anexa B**.

2.1 Cadrul tehnologic

2.1.1 Arhitecturi web moderne

Modelul tradițional client-server, în care browserul acționa exclusiv ca un consumator de **API** (interfață de programare a aplicațiilor)-uri **REST** (transfer de stare reprezentatională), nu mai răspunde integral cerințelor aplicațiilor **web** actuale. Așteptările privind performanța, experiența utilizatorului și vizibilitatea în motoarele de căutare au condus la apariția unor paradigme de **randare** mai flexibile, în care responsabilitățile sunt distribuite între client și server [6, 10].

În acest context, se disting mai multe strategii de **randare**:

- **CSR (randare pe client)**, care maximizează interactivitatea în browser;
- **SSR (randare pe server)**, orientată spre livrarea rapidă a conținutului inițial;
- **SSG (generare statică de pagini)**, potrivită pentru conținut stabil și public;
- **arhitecturi hibride**, care combină aceste tehnici în funcție de tipul rutei și de dinamica datelor.

Aplicația *Kookio* adoptă o astfel de arhitectură **full-stack** hibridă, în care **frontend**-ul și **backend**-ul sunt dezvoltate împreună, într-un flux unitar. Această decizie permite partajarea tipurilor și a mecanismelor de validare între straturi (ex.: **tRPC**+**Zod**), reducând riscul erorilor de integrare și sporind coerența aplicației [16].

2.1.2 Angular și randare server-side

Angular este un **framework** matur, construit în jurul limbajului **TypeScript** și orientat către dezvoltarea de aplicații **web** scalabile și bine structurate [7]. Deși utilizat inițial preponderent în scenarii de **CSR**, ecosistemul **Angular** a evoluat semnificativ în direcția suportului pentru **SSR**.

În cadrul proiectului *Kookio*, acest suport este furnizat de **Analog**, un **meta-framework** care extinde **Angular** cu funcționalități full-stack, precum:

- **SSR** (randare pe server),
- rutare bazată pe structură de fișiere,
- integrarea logicii de server în aceeași aplicație [6].

Randare pe server permite generarea **HTML**-ului inițial înainte ca aplicația să ajungă în browser, îmbunătățind atât timpul de afișare perceput, cât și compatibilitatea cu motoarele de căutare [10]. Pentru o aplicație orientată către conținut și uz casnic, precum *Kookio*, aceste aspecte devin relevante încă din primele interacțiuni ale utilizatorului.

2.1.3 Monorepo și orchestrare cu Nx

Gestionarea complexității unui proiect **full-stack** presupune o organizare clară a codului și a proceselor de build. În acest scop, proiectul *Kookio* este structurat ca **monorepo**, utilizând instrumentul **Nx** [9].

Această abordare permite gruparea într-un singur **repository** a aplicației **web** și a **bibliotecii** (**software**) asociate, fiecare cu responsabilități bine delimitate:

- interfața utilizator (**UI**) și logica server-side,
- accesul la date și modelele definite prin **ORM**,
- validarea contractelor de date,
- utilitare și date *mock* reutilizabile.

Nx adaugă peste această structură mecanisme de **execuția incrementală a sarcinilor**, **caching** și izolare logică între proiecte. Astfel, modificările locale pot fi propagate eficient, fără a afecta întregul sistem, un avantaj esențial în dezvoltarea iterativă a aplicațiilor **full-stack**.

2.1.4 tRPC și partajarea tipurilor

Un element central al arhitecturii [PAN](#) îl reprezintă comunicarea *type-safe* dintre client și server. În acest scop, proiectul utilizează [tRPC](#), un framework care elimină necesitatea definirii manuale a contractelor [API](#) [16].

Procedurile [tRPC](#) sunt declarate pe server și consumate direct în aplicația [Angular](#), beneficiind de:

- inferență completă a tipurilor [TypeScript](#),
- validarea automată a datelor de intrare prin scheme [Zod](#) [2],
- sincronizarea implicită a contractelor între straturi.

Această abordare simplifică semnificativ integrarea și creează un cadru robust pentru extinderea ulterioară a funcționalităților.

2.1.5 Persistență: Prisma și CockroachDB

Stratul de persistență al aplicației este construit pe baza [Prisma](#), un [ORM](#), și a sistemului de gestiune a bazelor de date [CockroachDB](#) [15, 3]. [CockroachDB](#) oferă un model [NewSQL](#) distribuit, compatibil cu protocolul [PostgreSQL](#), permițând atât consistență tranzacțională, cât și scalare orizontală [14].

[Prisma](#) joacă rolul de interfață declarativă între aplicație și baza de date, fiind responsabilă pentru:

- definirea schemei relaționale,
- generarea automată a clientului [TypeScript](#),
- gestionarea migrațiilor între versiuni ale structurii de date.

Această combinație oferă un echilibru între stabilitatea ecosistemului [SQL](#) și flexibilitatea necesară evoluției aplicației.

2.1.6 Testare automată: Vitest și Playwright

Asigurarea calității aplicației presupune validarea comportamentului acesteia la mai multe niveluri. În proiectul [Kookio](#), acest obiectiv este atins prin combinarea testare unitară ([unit](#)) cu [E2E](#).

Pentru testare unitară ([unit](#)) este utilizat [Vitest](#), ales pentru viteza de execuție și integrarea nativă cu ecosistemul [Vite](#) [12, 11]. Acesta permite testarea logicii de business și a componentelor într-un mediu controlat.

Testarea [E2E](#) și testare de integrare ([integration](#)) sunt realizate cu [Playwright](#), care simulează interacțiuni reale ale utilizatorului în browser [8]. Prin această abordare sunt validate fluxurile critice ale aplicației, de la încărcarea inițială a paginii până la comunicarea cu [API](#)-ul server-side.

2.1.7 Managementul configurației și mediilor

Configurarea aplicației este gestionată centralizat prin platforma de deployment [Vercel](#), care acționează ca sursă unică de adevăr pentru variabilele de mediu [18]. Această strategie permite separarea clară a mediilor de dezvoltare, testare și producție, reducând riscurile asociate gestionării locale a secretelor.

Pentru dezvoltarea locală, Vercel [CLI \(interfață de linie de comandă\)](#) este utilizat pentru sincronizarea temporară a variabilelor active, asigurând alinierea mediului local cu cel de deploy, fără introducerea fișierelor sensibile în [repository](#).

Capitolul 3

Implementare

3.1 Prezentare generală a implementării

Aplicația *Kookio* este implementată conform arhitecturii **PAN**, descrisă în Anexa B. Această secțiune prezintă structura concretă a proiectului, organizarea codului și fluxurile principale de execuție, din perspectiva implementării efective.

3.1.1 Structura monorepo-ului Nx

```
kookio/
├── web/                                # aplicație Analog SSR (Angular 20)
│                                       # cu client tRPC și module funcționale
├── web-e2e/                            # suită de teste Playwright pentru
│                                       aplicația web
├── libs/
│   ├── data-access/
│   │   ├── prisma-client              # schemă Prisma, migrații și
│   │   │                               scripturi dependente de mediu
│   │   └── prisma-validator          # scheme Zod generate din Prisma,
│   │                                       DTO-uri și validări suplimentare
│   └── shared/
│       ├── mock/                     # date de tip mock (ex. mockIngredients)
│       │                               pentru demonstrații și seed
│       └── utils/                    # utilitare TypeScript reutilizabile
│                                       între aplicații
├── dist/                              # artefacte de build generate de Nx
├── nx.json                            # configurație globală Nx și
│                                       valori implicite pentru target-uri
├── package.json                       # manifestul principal al proiectului
├── tsconfig.base.json                 # configurația de bază TypeScript
├── vercel.json                        # configurația proiectului Vercel
└── vitest.workspace.ts                # configurația Vitest la nivel de
workspace
```

Această organizare permite separarea clară a responsabilităților între aplicația **web**, **E2E** și **biblioteci (software)** reutilizabile, facilitând dezvoltarea incrementală și testarea independentă a componentelor [9].

3.1.2 Fluxul aplicației cap-coadă

În implementarea aplicației *Kookio*, fluxul principal de date urmează traseul: *Analog* (*UI și SSR (randare pe server)*) → *tRPC* (*apeluri de proceduri la distanță tipizate pentru TypeScript*) (*API type-safe*) → *Prisma* (*acces la date*) [6, 16, 15]. Această structură permite propagarea tipurilor *TypeScript* între client și server și integrarea directă a logicii de business în cadrul rutelor server-side [16].

3.2 Implementarea aplicației

Acest capitol descrie implementarea practică a aplicației *Kookio*, evidențiind modul în care conceptele prezentate în capitolele anterioare sunt transpuse într-un sistem funcțional. Sunt analizate structura proiectului, organizarea codului, fluxurile de date și deciziile tehnice care au stat la baza realizării aplicației.

3.2.1 Arhitecturi web moderne

Aplicarea practică a arhitecturilor *web* moderne presupune alegerea unei strategii de *randare* adaptate tipului de conținut și cerințelor de performanță. În proiectul *Kookio*, etapa de *randare* este abordată **hibrid**, combinând *SSR (randare pe server)* cu *SSG (generare statică de pagini)* acolo unde este posibil [6, 10].

- **SSR (randare pe server)** este utilizat pentru paginile dinamice, unde conținutul trebuie livrat rapid și în mod consistent indiferent de dispozitiv, serverul generând *HTML*-ul inițial [6, 10].
- **SSG (generare statică de pagini)** (prerender) este utilizat pentru rute stabile, care pot fi pre-generate la *build-time* (de exemplu pagini informative), reducând costul de execuție la runtime [6].
- **CSR (randare pe client)** rămâne relevant prin faptul că, după încărcarea inițială, interfața funcționează ca o aplicație interactivă în browser, iar actualizările sunt realizate fără reîncărcarea completă a paginii.

Această combinație permite un compromis eficient între performanță, flexibilitate și experiența utilizatorului, fără a separa artificial aplicația în proiecte distincte de *frontend* și *backend* [6, 10].

3.2.2 Structura proiectului și organizarea codului

Aplicația *Kookio* este implementată într-un *monorepo* gestionat cu *Nx*, care permite organizarea coerentă a aplicațiilor și *biblioteci (software)* reutilizabile [9]. Această abordare facilitează separarea responsabilităților și încurajează reutilizarea codului, menținând în același timp un control centralizat asupra configurațiilor și dependențelor.

Structura principală a proiectului este împărțită în:

- o aplicație [web](#) principală, responsabilă de [UI](#) și logica server-side;
- [biblioteci \(software\)](#) de acces la date, care conțin schema bazei de date și codul asociat [ORM \(mapare obiect-relațională\)](#) [15];
- [biblioteci \(software\)](#) partajate, utilizate pentru utilitare și date *mock*.

Această organizare permite dezvoltarea incrementală a aplicației și extinderea facilă a funcționalităților, fără a introduce dependențe circulare sau redundanță [9].

3.2.3 Angular și randare server-side

În cadrul proiectului *Kookio*, interfața este construită cu [Angular](#), iar suportul pentru [randare pe server](#) este obținut prin meta-framework-ul [Analog](#), care rulează peste runtime-ul [Nitro](#) [6, 10]. Această integrare permite ca aplicația să livreze [HTML](#) inițial generat pe server, iar ulterior să devină complet interactivă în browser.

Fluxul de execuție este compus din două etape:

- **Randare pe server (SSR):** la accesarea unei rute, serverul generează [HTML](#)-ul inițial și îl trimite către client, reducând timpul până la primul conținut vizibil și îmbunătățind compatibilitatea cu motoarele de căutare [6, 10].
- **Activarea interfeței în client:** după încărcare, codul [Angular](#) rulează în browser și preia controlul asupra interfeței, permițând interacțiuni dinamice și apeluri către [API](#) fără reîncărcări complete.

Un avantaj important al acestei abordări este faptul că rutele de aplicație și rutele de tip [API](#) coexistă în același proiect (ex.: [endpoint-uri](#) de tip [tRPC](#) și rute auxiliare precum `/api/health`), permițând implementarea unui sistem full-stack coerent, cu acces la baza de date realizat exclusiv pe server [6, 10, 16].

3.2.4 Gestionarea datelor cu Prisma și CockroachDB

Persistența datelor este realizată cu ajutorul [Prisma](#), configurat pentru a utiliza [CockroachDB](#) prin protocolul [PostgreSQL](#) [15, 3, 14]. [Prisma](#) oferă un model declarativ pentru definirea structurii bazei de date, precum și un client tipizat pentru interogarea acestuia din aplicație [15].

Schema bazei de date este definită într-un fișier central, din care [Prisma](#) generează automat:

- structura tabelor și a relațiilor,
- clientul de acces la date,
- tipuri [TypeScript](#) asociate modelelor [15].

Pentru modelul *Ingredient*, este utilizată o constrângere de unicitate compusă, care asigură faptul că un utilizator nu poate avea ingrediente duplicate cu același nume. Această regulă este exprimată direct în schema [Prisma](#) și este aplicată automat la nivelul bazei de date [15].

3.2.5 Comunicarea client–server cu tRPC

Pentru comunicarea dintre client și server este utilizat [tRPC](#) ([apeluri de proceduri la distanță tipizate pentru TypeScript](#)), un framework care permite definirea API-urilor tipizate fără a introduce un strat suplimentar de serializare manuală [16]. Prin această abordare, tipurile sunt partajate între [frontend](#) și [backend](#), eliminând discrepanțele dintre contractele de date [16].

Routerele [tRPC](#) sunt definite pe server și expun operații precum:

- crearea unui ingredient,
- listarea ingredientelor,
- actualizarea și ștergerea acestora.

Clientul [Angular](#) consumă aceste operații direct, beneficiind de verificare statică a tipurilor și de autocompletare în timpul dezvoltării [16]. Această integrare reduce semnificativ riscul de erori la rulare și crește productivitatea dezvoltatorului.

3.2.6 Randare server-side și integrarea backend-ului

Aplicația utilizează [randare pe server](#) prin [Analog](#), care rulează peste infrastructura [Nitro](#) [6, 10]. Această abordare permite definirea rutelor API și a logicii de server direct în cadrul aplicației [web](#), fără a necesita un [backend](#) separat [10].

Rutele server-side includ atât [endpoint-uri](#) de tip API, cât și [handler-e](#) pentru [tRPC](#) [16, 10]. Prin această integrare, aplicația poate executa interogări către baza de date direct pe server, în contextul fiecărei cereri [HTTP](#) [15].

Această arhitectură reduce latența percepută de utilizator și permite un control mai bun asupra fluxului de date și al securității aplicației.

3.2.7 Jurnalizare și observabilitate

Pentru monitorizarea comportamentului aplicației și facilitarea depanării, [Kookio](#) utilizează un mecanism de [jurnalizare \(logging\)](#) unitar, disponibil atât în context server-side, cât și în execuția locală, în sprijinul [observabilitate](#).

Jurnalizarea este realizată cu ajutorul bibliotecii [Pino](#) [1], un jurnal performant pentru aplicații Node.js, ales pentru consumul redus de resurse și integrarea facilă în medii server-side moderne. Această abordare permite înregistrarea coerentă a evenimentelor relevante din fluxul de execuție, fără a afecta semnificativ performanța aplicației.

3.2.8 Fluxul de execuție al unei cereri

Pentru a înțelege integrarea completă a componentelor prezentate anterior, este utilă analiza fluxului de execuție al unei cereri tipice în aplicația [Kookio](#), de la inițierea acesteia în browser până la accesul la datele persistente.

Atunci când un utilizator accesează o rută a aplicației, cererea [HTTP](#) este preluată de runtime-ul server-side furnizat de [Analog](#), bazat pe infrastructura [Nitro](#) [6, 10]. În acest stadiu, aplicația [Angular](#) este executată pe server, iar structura inițială a paginii este randată sub formă de [HTML](#) [6].

În timpul [randare pe server](#), componentele [Angular](#) pot declanșa apeluri către proceduri [tRPC](#), definite în stratul server-side al aplicației [16]. Aceste proceduri reprezintă punctul de intrare pentru logica de business și realizează validarea datelor de intrare prin scheme [Zod](#), generate din modelul [Prisma](#) [2, 15, 4].

Procedurile [tRPC](#) utilizează [Prisma Client](#) pentru a interacționa cu baza de date [CockroachDB](#), executând operații de citire sau scriere într-un context tranzacțional sigur [15, 3]. Rezultatele interogărilor sunt returnate către nivelul [tRPC](#), apoi propagate către componentele [Angular](#) care participă la randarea inițială a paginii [16].

După finalizarea procesului de [randare pe server](#), [HTML](#)-ul rezultat este trimis către browser. În paralel, aplicația [Angular](#) este descărcată și inițializată pe client, unde are loc procesul de [hidratare \(hydration\)](#), prin care structura [DOM](#) generată pe server este reutilizată și îmbogățită cu interactivitate [6].

Apelurile ulterioare inițiate din interfața utilizatorului (de exemplu, adăugarea sau actualizarea unui ingredient) sunt realizate direct din browser către procedurile [tRPC](#), fără a mai necesita o [randare](#) completă pe server [16]. Astfel, aplicația combină avantajele [randare pe server](#) pentru încărcarea inițială cu flexibilitatea și performanța interacțiunilor pe client.

Acest flux de execuție unitar demonstrează beneficiile arhitecturii full-stack adoptate în proiectul *Kookio*, în care [frontend](#)-ul, [backend](#)-ul și stratul de persistență funcționează coerent într-un singur ecosistem [TypeScript](#) [9, 6, 16, 15].

3.2.9 Testare și validare

Procesul de validare a aplicației include atât [testare unitară \(unit\)](#), cât și [testare de integrare \(integration\)](#), fiind integrat în fluxul de dezvoltare al [monorepo](#)-ului [Nx](#) [9].

Pentru [testare unitară \(unit\)](#) este utilizat [Vitest](#), ales pentru integrarea sa nativă cu [Vite](#) și compatibilitatea cu ecosistemul [Angular](#) utilizat de [Analog](#) [12, 11, 6]. Testele acoperă componentele [UI](#), serviciile și logica de business, rulând într-un mediu controlat care permite izolarea dependențelor și verificarea comportamentului funcțiilor critice.

Testarea [E2E](#) și [testare de integrare \(integration\)](#) sunt realizate cu [Playwright](#), care simulează interacțiunea utilizatorului cu aplicația în browser [8]. Această verificare este deosebit de relevantă în contextul [SSR \(randare pe server\)](#), întrucât permite validarea fluxurilor complete, de la generarea inițială a paginii pe server până la interacțiunea cu datele persistente prin [API](#)-ul [tRPC](#) [6, 16].

Combinarea testelor [testare unitară \(unit\)](#), de [testare de integrare \(integration\)](#) și [E2E](#) contribuie la identificarea timpurie a defectelor și la creșterea stabilității aplicației, asigurând un comportament predictibil al sistemului în diferite scenarii de utilizare.

3.2.10 Dezvoltare și deploy

Aplicația este dezvoltată local folosind un mediu care reproduce cât mai fidel configurația de producție. Variabilele de mediu sunt gestionate centralizat prin platforma [Vercel](#), iar pentru dezvoltarea locală acestea sunt sincronizate cu ajutorul Vercel [CLI](#) [18].

Deploy-ul aplicației este realizat pe platforma [Vercel](#), care oferă suport nativ pentru aplicații server-side și integrare continuă (CI). Această abordare permite rularea aplicației în medii distincte (dezvoltare, testare, producție), cu baze de date separate și configurări independente.

În practică, generarea [Prisma Client](#) trebuie realizată pe platforma țintă de execuție ([Linux](#) în cazul [Vercel](#)), deoarece [binare \(binaries\)](#) Prisma sunt dependente de sistemul de operare și de biblioteca [SSL](#) utilizată. Utilizarea unui client generat pe o mașină locală poate conduce la incompatibilități de tip [darwin-arm64](#) vs. [rhel-openssl-3.0.x](#). Soluția adoptată constă în generarea [Prisma Client](#) în cadrul pipeline-ului de build și deploy.

Capitolul 4

Metodologie și evaluare

4.1 Setare experimentală

Mediu: [Vercel](#) (preview/prod), [CockroachDB](#) (cluster ...).

Build-uri: [Analog](#) pentru [SSR \(randare pe server\)](#) (browser și server).

Date: fixtures deterministe ([mockIngredients](#)).

4.2 Indicatori și proceduri

4.2.1 Timp rulare [CI \(integrare continuă\)](#)

Comparăm monolit ([workflow](#) unic) vs. modular ([workflow-uri](#) reutilizabile + [composite actions](#)).

4.2.2 Latența [DB \(bază de date\)](#)

Rulăm `bench:preview` cu $N = 100$; raportăm medie, [p50](#), [p95](#); notăm *cold/warm*.

4.2.3 Fiabilitate [E2E \(testare cap-coadă\)](#)

Rată de trecere [Playwright](#) pe k rulări, pe matrici de browser (Chromium/WebKit/Firefox).

4.3 Rezultate

[CI](#): 4 min $\rightarrow$ 3 min (mediană).

[CockroachDB](#): [avg](#) = 2.55 ms, [p50](#)=2.23 ms, [p95](#)=3.52 ms.

[Redis](#): [avg](#) = 1.2 ms, [p50](#)=0.98 ms, [p95](#)=1.52 ms.

[E2E](#): 100% rata de trecere.

4.4 Discuție și limitări

Rezultate din mediu de *preview*; nu includ trafic concurent. Generalizarea cere teste de încărcare și măsurători în producție.

Capitolul 5

Concluzii

Lucrarea de față a avut ca obiectiv proiectarea și implementarea unei aplicații **web full-stack** moderne, menită să demonstreze integrarea coerentă a tehnologiilor actuale din ecosistemul **TypeScript** într-o arhitectură unitară. Proiectul *Kookio* a fost conceput atât ca aplicație funcțională, cât și ca studiu de caz pentru analiza arhitecturilor **web** bazate pe **randare hibridă** utilizate în contexte reale.

Adoptarea arhitecturii **PAN** a permis realizarea unui flux cap-coadă tipizat, de la interfața utilizatorului până la nivelul de persistență al datelor. Această abordare reduce discrepanțele dintre **frontend** și **backend**, elimină duplicarea contractelor de date și facilitează dezvoltarea incrementală a funcționalităților. În acest context, arhitectura **PAN** se conturează ca un model viabil pentru aplicații **web full-stack** moderne.

Utilizarea **Angular** împreună cu **Analog** a permis implementarea unei aplicații cu **SSR** (randare pe server) și **hidratare (hydration)** pe client. Această combinație îmbunătățește performanța percepută și compatibilitatea cu motoarele de căutare. Integrarea logicii de server în aceeași aplicație cu interfața utilizatorului simplifică arhitectura generală și reduce necesitatea unui **backend** separat, fără a compromite separarea responsabilităților.

Pentru accesul la date, combinația dintre **Prisma** și **CockroachDB** oferă un model de persistență robust, compatibil cu standardele **SQL** și pregătit pentru scenarii de scalare viitoare. Definirea declarativă a schemei bazei de date, alături de generarea automată a **Prisma Client**, contribuie la creșterea siguranței, lizibilității și mentenabilității codului.

Procesul de testare a evidențiat importanța combinării mai multor niveluri de validare. Testele **testare unitară (unit)** cu **Vitest** asigură corectitudinea logicii interne a aplicației. În completare, testele **E2E** și **testare de integrare (integration)** realizate cu **Playwright** validează funcționarea sistemului din perspectiva utilizatorului final, inclusiv în contextul **SSR**.

Din punctul de vedere al obiectivelor inițiale, proiectul îndeplinește cerințele funcționale ale unui **MVP**. Aplicația oferă funcționalități de bază pentru gestionarea ingredientelor și demonstrează fezabilitatea arhitecturii propuse. Structura modulară a **monorepo**-ului gestionat cu **Nx** permite extinderea ulterioară a aplicației fără modificări structurale majore.

În ansamblu, studiul realizat prin aplicația *Kookio* arată că arhitectura **PAN** poate fi utilizată ca model coerent pentru dezvoltarea aplicațiilor **web full-stack** moderne. Aceasta

oferă atât valoare practică, cât și un cadru solid pentru analiză academică și dezvoltare ulterioară.

Direcții viitoare

Direcțiile viitoare de dezvoltare includ extinderea mecanismelor de [AuthN](#) și [AuthZ](#), optimizarea performanței prin introducerea unor strategii de [caching](#) și integrarea unor servicii inteligente pentru recomandarea rețetelor.

Aceste extensii pot fi realizate incremental, fără a afecta fundamentele arhitecturale prezentate în această lucrare.

Anexa A

Anexă: Redactarea lucrării în L^AT_EX

A.1 Instrumente pentru redactare

Lucrarea a fost redactată cu motorul [LuaLaTeX](#), folosind fonturi [Unicode](#) și pachetul [biblatex](#) pentru gestionarea bibliografiei. Structura este modulară, fiecare capitol fiind un fișier separat (`.tex`) compilat prin scripturi automate de [build](#). Imaginile sunt salvate în `images/`, iar citările se realizează cu `\cite{}`.

A.2 Compatibilitate multiplatformă

Există scripturi automate pentru [Linux](#) (`build.sh`), [Windows](#) (`build.bat`) și [PowerShell](#) (`build.ps1`) care generează [PDF \(format de document portabil\)](#) în directorul `dist/` și afișează eventualele erori de compilare.

Notă: Mediul [TreeVerb](#) utilizează fontul [FreeMono](#) pentru suport complet al caracterelor [Unicode](#) de tip box-drawing (U+2500–U+257F).

Anexa B

Kookio: Manifest Arhitectural PAN (Prisma–Analog–Nx)

B.1 Viziune generală

Kookio este o aplicație **full-stack** dezvoltată în **TypeScript**, bazată pe arhitectura **PAN** — un cadru scalabil care integrează **Prisma** pentru acces la date, **Analog** pentru **randare hibridă** și **Nx** pentru orchestrare **monorepo**. Această abordare reprezintă o alternativă la stack-uri consacrate (**JAMstack**, **MEAN**), fiind adaptată ecosistemului **Angular** și cerințelor enterprise.

B.2 Componentele arhitecturii

- **Prisma** – ORM cu suport pentru **PostgreSQL** și extensii **NewSQL** (**CockroachDB**).
- **Analog** – meta-framework pentru **Angular** cu **SSR** (randare pe server), **Angular Signals** și **tRPC** (apeluri de proceduri la distanță tipizate pentru **TypeScript**), urmate de **hidratare** (hydration) în client.
- **Nx** – orchestrator modular care optimizează **build**-urile și testele în context **monorepo**.

B.3 Structura fizică a proiectului

Structura completă a **monorepo**-ului este ilustrată în § 3.1.1 din capitolul 3.

B.4 Avantaje comparative

Caracteristică	JAMstack	MEAN	PAN (Kookio)
SSR / randare hibridă	Parțial (Next.js)	Nu	Da (Analog + SSR)
Type-safety end-to-end	Parțial	Nu (Mongo/JS)	Da (TypeScript + Prisma + Angular)
Structurare modulară	Slabă	Medie	Excelentă (Nx + biblioteci)
Acces la date	Headless CMS/REST	REST/Mongoose	ORM modern (Prisma)
Adaptabilitate monorepo	Nu	Nu	Da

Tabela B.1: Comparatie între arhitectura PAN și alte stack-uri moderne

B.5 Direcții de viitor

Direcțiile viitoare de dezvoltare sunt prezentate în § 5, unde sunt discutate extensiile funcționale și tehnice ale aplicației.

B.6 Concluzie

Arhitectura PAN sintetizează cele mai bune practici moderne din ecosistemul TypeScript, oferind o experiență unificată full-stack și o bază solidă pentru aplicații educaționale, alimentare sau de lifestyle, precum viziunea Kookio. PAN nu reprezintă un standard formal, ci un model arhitectural derivat din integrarea unor tehnologii mature, validat prin implementarea aplicației Kookio.

Glosar de termeni

Analog meta-framework full-stack pentru [Angular](#), cu suport [SSR](#) și rutare pe fișiere; folosit pentru aplicații [web](#)

Angular platformă și framework pentru aplicații [web](#) (SPA/[SSR](#))

Nitro runtime universal (UnJS) pentru rute server și randare pe server ([SSR](#))

Nx sistem de build și orchestrare pentru monorepo-uri JavaScript/TypeScript

Prisma bibliotecă [ORM](#) pentru TypeScript/Node.js

Zod bibliotecă de validare pe bază de schemă pentru TypeScript/JavaScript

Angular Signals primitive reactive introduse în Angular pentru modelarea stării și propagarea schimbărilor cu granularitate fină

avg medie aritmetică a unui set de valori; în SQL corespunde funcției **AVG**, care (în implementările uzuale) ignoră valorile NULL. Se folosește în raportare împreună cu percentile precum [p50](#) (mediană) și [p95](#)

backend stratul server-side care implementează logica de business, expune [API](#)-uri și mediază accesul la date (ex.: [Prisma](#) către [CockroachDB](#)). Poate rula ca [SSR](#), funcții [serverless](#)/[Edge Functions](#) sau servicii dedicate. În acest proiect, backend-ul este integrat în [Analog](#) (Nitro routes și routere [tRPC](#)).

biblatex pachet $\text{\LaTeX}$ pentru gestionarea bibliografiilor și citărilor cu backend-uri moderne (de ex. **biber**)

bibliotecă (software) colecție reutilizabilă de funcții/clase pe care aplicația o apelează explicit pentru a rezolva probleme specifice. Spre deosebire de un [framework](#), o bibliotecă nu inversează controlul și nu dictează structura aplicației. Exemple în această lucrare: [Zod](#) (validare pe bază de schemă), [Prisma](#) (client [ORM](#)), [Vitest](#) (testare unitară/integration), [Playwright](#) (testare [E2E](#)).

bibliotecă standard colecția de [API](#) de bază oferite implicit de un limbaj sau runtime, disponibilă fără dependențe externe. Include tipuri, structuri de date, I/O și utilitare fundamentale. În contextul acestei lucrări: bibliotecile standard ale JavaScript/TypeScript (tipuri și obiecte de bază) și ale runtime-ului Node.js (ex. **fs**, **path**). Se deosebește de o [bibliotecă \(software\)](#) terță și de un [framework](#) care impune inversiunea controlului.

binar (binary) fișier executabil sau bibliotecă nativă generată prin compilare, specifică unei platforme (sistem de operare și arhitectură [CPU](#)); include executabile și biblioteci partajate (.so/.dylib/.dll). În contextul [Kookio](#), [Prisma](#) folosește componente native (engines) care trebuie compatibile cu bibliotecă OpenSSL a sistemului (de ex. `rhel-openssl-3.0.x`); de aceea clientul este generat pe platforma țintă (ex.: pe [Linux](#) în timpul deploy-ului pe [Vercel](#)).

build proces automatizat care transformă codul sursă și resursele (ex.: TypeScript, CSS, imagini) în artefacte optimizate pentru execuție (server/[Edge Functions](#)/browser): transpilare/compilare, bundling, minificare, tree-shaking, code splitting, generare de sourcemaps și, când e cazul, [prerender](#) ([SSG](#)). Rezultatul este publicat de regulă în `dist/`. Se deosebește de *deploy*, care publică artefactele într-un mediu țintă.

build-time faza de livrare în care codul este transpilat, agregat și optimizat înainte de publicare; aici pot rula sarcini precum [prerender](#) ([SSG](#)), generarea de asset-uri și verificări statice. Se deosebește de execuția per cerere pe server ([SSR](#)) sau în browser ([CSR](#)). În acest proiect, pașii critici de build sunt gestionați de [Vite](#) și [Analog](#).

caching tehnici și politici de păstrare temporară a rezultatelor calculului sau ale răspunsurilor [API](#) pentru a reduce latența și costurile de execuție. În contextul [SSR/SSG](#) și al platformelor moderne (ex. [Vercel](#)), caching-ul poate fi aplicat la nivel de browser, rețea (CDN/edge) și aplicație (cache în memorie sau persistent). Include mecanisme de expirare și revalidare (ex. invalidare la scriere, la timp sau la eveniment) și strategii de tip *stale-while-revalidate* pentru servirea rapidă a conținutului cu actualizare în fundal.

CockroachDB sistem de gestiune a bazelor de date distribuit ([NewSQL](#)), compatibil la nivel de protocol cu [PostgreSQL](#), cu replicare și partiționare automate pentru reziliență și scalare orizontală

composite action tip de acțiune reutilizabilă în [GitHub](#) Actions care grupează mai mulți pași (`run/uses`) într-o singură acțiune; se distribuie ca director cu fișier `action.yml`, acceptă *inputs*, *outputs* și *secrets* și poate fi apelată dintr-un [workflow](#) prin `uses::`. Utilă pentru a elimina duplicarea (DRY) în pipeline-urile [CI/CD](#); limitări uzuale: nu rulează în containere proprii și nu pot defini *services*.

darwin-arm64 etichetă de platformă (target) utilizată la generarea [binare \(binaries\)](#) pentru ecosistemul Node.js pe macOS (kernel *Darwin*) cu arhitectură ARM64 (Apple Silicon). În [Prisma](#), selectarea acestui target indică faptul că *engines* native sunt construite pentru dezvoltare locală pe macOS ARM și nu sunt compatibile cu [Linux](#) de producție; diferențele de [CPU](#) și biblioteci de sistem pot produce erori la runtime.

arbore DOM reprezentarea ierarhică a unui document (de obicei HTML) sub formă de noduri și relații părinte–copil, conform [DOM](#)

Edge Functions funcții executate în rețeaua de *edge* (aproape de utilizator), utile pentru latență redusă, personalizare și pre-/post-procesare a răspunsurilor [SSR](#); pot rula cu constrângeri mai stricte (timp CPU, memorie) față de funcțiile serverless clasice

endpoint Punct final (URL) la care aplicația răspunde unei cereri [HTTP](#). În această lucrare, termenul acoperă atât rute [API](#) clasice (ex.: `/api/health`), cât și proceduri [tRPC](#) expuse sub o rută agregatoare (ex.: `/api/trpc`). Un endpoint definește metoda, schema de intrare/ieșire și semantica răspunsului; este procesat de un [handler](#).

framework set de convenții, componente și unelte care oferă o structură impusă pentru dezvoltarea de aplicații. Un *framework* controlează fluxul principal al aplicației (inversiune a controlului), definește puncte de extensie și impune un mod coerent de organizare a codului; spre deosebire de o bibliotecă, pe care aplicația o apelează explicit. Exemple în lucrare: [Angular](#); [Analog](#) funcționează ca *meta-framework* peste [Angular](#) pentru [SSR](#) și rute server.

FreeMono font monospațiat din familia GNU FreeFont, cu suport Unicode extins (inclusiv caractere *box-drawing* U+2500–U+257F)

frontend stratul aplicației care rulează în browser și gestionează [UI](#)-ul, logica de interacțiune și actualizările în [CSR](#). În arhitecturi hibride consumă HTML-ul generat prin [SSR/SSG](#) și trece prin [hidratare \(hydration\)](#). Comunică cu [backend](#)-ul prin [API](#) (ex.: [tRPC](#)). În acest proiect, frontend-ul este implementat cu [Angular](#) via [Analog](#).

full-stack abordare în care aceiași dezvoltatori/proiect acoperă cap-coadă [frontend](#)-ul și [backend](#)-ul, inclusiv modelarea datelor și pipeline-urile de [CI/CD](#). Beneficii: tipuri partajate, coerență arhitecturală și viteză de livrare. În această lucrare, modelul full-stack este realizat prin arhitectura [PAN](#) într-un [monorepo](#) Nx.

GitHub platformă de găzduire și colaborare pentru proiecte software bazate pe sistemul de versiuni Git; oferă găzduire de [repository-uri](#), mecanisme de colaborare (pull requests, code review), management de *issues* și [CI/CD](#) prin GitHub Actions; utilizată frecvent pentru automatizări și publicare.

GitLab platformă de găzduire și colaborare pentru proiecte software bazate pe Git, disponibilă ca serviciu cloud sau instalare self-hosted; integrează nativ [CI/CD](#) (GitLab CI/CD), management de *issues*, *merge requests* și registru de containere; alternativă la [GitHub](#) cu accent pe lanțul DevOps unificat.

handler Funcție care primește o cerere (*request*) și produce un răspuns (*response*). În [Nitro/Analog](#) un handler procesează rute server (inclusiv [SSR](#)) sau endpoint-uri [API](#); în [tRPC](#), *resolver*-ele procedurilor acționează ca handler-e pentru apeluri tipizate.

hidratare (hydration) procesul prin care codul din client atașează comportament interactiv la *markup*-ul generat prin [SSR](#); permite trecerea de la randare inițială pe server la interacțiuni [CSR](#) fără reîncărcarea paginii

testare de integrare (integration) nivel de testare care verifică interacțiunea dintre module/straturi (ex. [UI](#)+[API](#), [tRPC](#)+[Prisma](#)); în această lucrare este realizat predominant cu [Playwright](#) (inclusiv scenarii de integrare fără [UI](#) complet, folosind fixtures și acces la [API](#) atunci când este util)

Kookio aplicație [web](#) full-stack folosită ca studiu de caz în această lucrare pentru gestionarea inventarului de ingrediente în arhitectura [PAN](#)

Linux nucleu de sistem de operare și familie de distribuții GNU/Linux folosite pe scară largă în cloud, servere și desktop

jurnalizare (logging) colectarea și înregistrarea evenimentelor aplicației (mesaje structurate pe niveluri, corelare prin identificatori, export către sisteme de agregare) pentru depanare și observabilitate

LuaLaTeX motor $\text{\LaTeX}$ bazat pe LuaTeX, cu suport nativ pentru Unicode și fonturi OpenType

meta-framework framework care extinde un framework de bază pentru capacități full-stack (ex. rutare pe fișiere, [SSR/SSG](#), integrare cu [tRPC](#)). În această lucrare, [Analog](#) extinde [Angular](#) peste runtime-ul [Nitro](#).

Mongoose bibliotecă de *modelare a datelor* pentru *MongoDB* în ecosistemul Node.js; oferă *scheme*, validare, relații și *middleware/hooks* pentru gestionarea logicii de persistență. Este frecvent folosită în cadrul stack-ului [MEAN](#)

monorepo strategie de organizare a codului în care mai multe aplicații și biblioteci coexistă într-un singur depozit, cu orchestrare unitară a build-urilor, testelor și [CI/CD](#); în contextul lucrării este gestionat cu Nx

NewSQL clasă de sisteme de gestiune a bazelor de date care păstrează modelul relațional și limbajul [SQL](#), oferind în același timp scalare orizontală și consistență ACID într-un mediu distribuit; multe implementări sunt compatibile la nivel de protocol cu [PostgreSQL](#) (de ex. [CockroachDB](#))

Next.js framework React orientat pe randare hibridă ([SSR/SSG](#)), frecvent asociat cu [JAMstack](#)

observabilitate capacitatea unui sistem de a-și expune starea internă prin semnalizare (loguri, metrice, trasabilitate) pentru analiză și diagnostic

Pino bibliotecă de logare pentru Node.js, orientată pe performanță (JSON logs, output low-overhead)

Playwright bibliotecă pentru automatizarea browserelor și testare [E2E](#), cu suport pentru Chromium, Firefox și WebKit, rulare headless/GUI și unelte precum trace viewer

PostgreSQL sistem de gestiune a bazelor de date relaționale open-source, cu suport extins pentru standardul [SQL](#), tranzații ACID și extensibilitate prin tipuri și funcții definite de utilizator

PowerShell limbaj de scriptare și mediu de automatizare Microsoft, utilizat pentru task-uri de build și DevOps pe Windows

prerender generarea în prealabil a HTML-ului pentru o rută la [build-time](#), rezultând fișiere statice ce pot fi servite direct (ex. din CDN). Este tehnică specifică [SSG](#), utilă pentru conținut stabil; poate coexista cu rute [SSR](#) și interacțiuni [CSR](#). În [Analog](#), rutele stabile pot fi prerenderizate pentru latență minimă și cost redus.

Prisma Client bibliotecă TypeScript generată automat de [Prisma](#) pe baza fișierului `schema.prisma`; oferă un API tipizat pentru operații [SQL](#) (CRUD, filtre, relații, tranzacții) asupra modelelor definite. Utilizează motoare native (Query Engine) distribuite ca [binare \(binaries\)](#) dependente de platformă (ex.: [darwin-arm64](#), [rhel-openssl-3.0.x](#)), motiv pentru care se generează pe mediul țintă de deploy; se invocă prin `npx prisma generate`.

randare procesul prin care o aplicație generează HTML (și actualizări ale interfeței) din cod și date. În web-ul modern poate avea loc pe server ([SSR](#)), la [build-time](#) prin [SSG/prerender](#), sau în browser ([CSR](#)); de regulă, este urmată de [hidratare \(hydration\)](#) pentru interactivitate. În [Analog](#), strategia de randare poate fi aleasă per rută.

randare hibridă abordare care combină [SSR](#), [SSG/prerender](#) și [CSR](#) în aceeași aplicație, alegând mecanismul optim per rută/scenariu. Avantaje: timp de răspuns inițial redus și compatibilitate ridicată cu motoarele de căutare; cost redus pe rute statice; interactivitate ridicată în client. În acest proiect, [Analog](#) permite configurarea hibridă la nivel de rute.

Redis magazin de date *in-memory* tip cheie-valoare, cu structuri avansate (liste, seturi, hash-uri, mulțimi ordonate, *streams*) și latență foarte mică; folosit frecvent pentru [caching](#), stocare sesiuni, cozi/mesagerie (Pub/Sub, Streams), rate-limiting și *leaderboards*. Suportă persistență opțională (RDB/AOF), replicare, *cluster* pentru partiționare și, de regulă, completează o bază [SQL/DB](#) principală în arhitecturi microservicii și [serverless/Edge Functions](#).

repository spațiu logic pentru stocarea codului sursă, a istoricului de versiuni și a metadatelor; poate fi local sau la distanță (remote, ex.: GitHub/GitLab). În această lucrare, monorepo-ul reunește mai multe proiecte într-un singur repository.

rhel-openssl-3.0.x etichetă de platformă pentru distribuții [Linux](#) compatibile Red Hat Enterprise Linux care folosesc biblioteca OpenSSL 3.0.x. În [Prisma](#), acest target este folosit în `binaryTargets` pentru a genera [binare \(binaries\)](#) compatibile cu mediile de execuție tipice de producție (ex.: containere bazate pe *glibc*); alegerea corectă elimină erorile de tip `wrong platform for query engine` la deploy.

serverless model operațional în care alocarea resurselor și scalarea sunt gestionate de platformă; dezvoltatorul furnizează funcții sau servicii ce pornesc la cerere, cu facturare pe utilizare; folosit adesea pentru [API-uri](#), [SSR](#) și joburi programate

TreeVerb mediu *verbatim* personalizat (bazat pe `fvextra/fancyvrb`) pentru randarea arborilor/structurilor de fișiere; folosește [FreeMono](#) pentru simboluri *box-drawing*

TypeScript superset tipizat al limbajului JavaScript care introduce tipuri statice și se transpilează la JavaScript

Unicode standard de codificare a caracterelor care oferă o mapare unică pentru simboluri folosite în scrieri și tehnică (ex. caractere *box-drawing*)

testare unitară (unit) nivel de testare axat pe componente izolate (funcții, clase, componente [UI](#)), cu dependențe înlocuite (*test doubles*); în lucrare este rulat predominant cu [Vitest](#)

Vercel platformă cloud pentru build, preview și deploy de aplicații [web](#), cu integrare [CI/CD](#), variabile de mediu per mediu și suport pentru [SSR](#), funcții [serverless](#) și [Edge Functions](#)

Vite instrument de build și server de dezvoltare pentru aplicații [web](#) moderne, bazat pe module ECMAScript, cu pornire foarte rapidă și *hot module replacement* (HMR); stă la baza [Vitest](#) și este folosit de [Analog](#) pentru [SSR](#) și build

Vitest framework de testare rapid pentru ecosistemul Vite, orientat spre teste unitare și de integrare, cu API în mare parte compatibil Jest și suport nativ pentru TypeScript

web ecosistemul de resurse *hipertext* accesibile prin Internet (World Wide Web); în lucrare, „aplicație web” desemnează aplicație accesată din browser, cu randare [SSR/SSG/CSR](#)

Windows sistem de operare Microsoft utilizat preponderent pe desktop și laptop, frecvent în mediile enterprise

workflow configurație declarativă care descrie o succesiune de pași automatizați (build, testare, lint, deploy) declanșați de evenimente (ex.: push, pull request, programare). În ecosisteme precum [GitHub Actions](#) și [GitLab CI](#), un workflow este compus din job-uri și pași, poate folosi matrici, cache, artefacte, secrete/variabile și reguli de concurență. În acest proiect, workflow-urile orchestrează [CI/CD](#) și aliniaza execuția la țintele [Nx](#).

Lista de acronime

API interfață de programare a aplicațiilor

AuthN autentificare

AuthZ autorizare

CD livrare/implementare continuă

CI integrare continuă

CI/CD integrare continuă și livrare/implementare continuă

CLI interfață de linie de comandă

CMS sistem de gestionare a conținutului

CPU unitate centrală de procesare

CSR randare pe client

DB bază de date

DOM model al obiectelor documentului

DTO obiect de transfer de date

E2E testare cap-coadă

GPU unitate de procesare grafică

HTML HyperText Markup Language (limbaj de marcare hipertext)

HTTP protocol de transfer hipertext

HTTPS protocol de transfer hipertext securizat

JAMstack JavaScript, API-uri și Markup

MEAN MongoDB–Express–Angular–Node.js

MFA autentificare multi-factor

MVP produs minim viabil

ODM modelare a datelor orientată pe obiecte

ORM mapare obiect-relațională

p50 percentila 50 (mediană)

p95 percentila 95

PAN Prisma-Analog-Nx

PDF format de document portabil

RAM memorie cu acces aleator

REST transfer de stare reprezentatională

SQL limbaj de interogare structurat

SSG generare statică de pagini

SSL strat de socketuri securizate

SSO autentificare unică

SSR randare pe server

TLS securitatea stratului de transport

tRPC apeluri de proceduri la distanță tipizate pentru TypeScript

UI interfață de utilizator

URL localizator uniform de resurse

VCS sistem de control al versiunilor

Bibliografie

- [1] Autorii Pino, *Pino — Logger pentru Node.js*, Documentația Pino, Documentație oficială Pino; citată pentru logarea centralizată utilizată atât în backend, cât și în frontend., 2025, URL: <https://getpino.io/> (accesat în 30.9.2025).
- [2] Autorii Zod, *Documentația Zod*, Documentația Zod, Documentație oficială Zod; citată pentru validarea schemelor utilizate împreună cu tipurile generate de Prisma., 2025, URL: <https://zod.dev> (accesat în 15.6.2025).
- [3] Cockroach Labs, *Documentația CockroachDB*, Documentația CockroachDB, Documentație oficială CockroachDB; citată în planul de dezvoltare pentru analiza scalabilității orizontale., 2025, URL: <https://www.cockroachlabs.com/docs/stable/> (accesat în 18.10.2025).
- [4] Omar Dulaimi și colaboratorii, *Generatorul Prisma Zod*, Depozitul GitHub, Versiunea 1.27.7; citată pentru generarea automată a schemelor de validare Zod pe baza modelelor Prisma., 2025, URL: <https://github.com/omar-dulaimi/prisma-zod-generator> (accesat în 18.10.2025).
- [5] Comunitatea JSON Schema, *JSON Schema — Învăță (Learn)*, Documentația JSON Schema, Resursă oficială JSON Schema; citată pentru validarea externă și interoperabilitatea menționată în planul de dezvoltare., 2025, URL: <https://json-schema.org/learn/> (accesat în 15.6.2025).
- [6] Dezvoltatorii AnalogJS, *Analog – Meta-framework full-stack pentru Angular*, Documentația Analog, Documentație oficială Analog; citată pentru SSR, rutare bazată pe fișiere și integrarea Nitro/tRPC în aplicațiile Angular., 2025, URL: <https://analogjs.org> (accesat în 14.9.2025).
- [7] Echipa Google Angular, *Prezentare generală a semnalelor (Signals Overview)*, Documentația Angular, Documentație oficială Angular; citată pentru modelul de reactivitate (Signals) utilizat în componente independente și în contextul randării pe server (SSR)., 2025, URL: <https://angular.dev/guide/signals> (accesat în 15.6.2025).
- [8] Echipa Microsoft Playwright, *Testarea cu Playwright — Ghid introductiv*, Documentația Playwright, Documentație oficială Playwright; citată pentru testarea end-to-end adoptată în perioada iunie–iulie 2025., 2025, URL: <https://playwright.dev/docs/intro> (accesat în 10.7.2025).
- [9] Echipa Nrwl, *Ce este Nx?*, Documentația Nx, Documentație oficială Nx; citată pentru orchestrarea monorepo-ului, rulările afectate, caching și graful de proiect., 2025, URL: <https://nx.dev/getting-started/intro> (accesat în 15.6.2025).

- [10] Echipa UnJS, *Nitro: Motorul de server pentru web*, Documentația Nitro, Documentație oficială Nitro; citată pentru execuția pe server, rutele API și mediile de implementare compatibile cu Analog SSR., 2025, URL: <https://nitro.unjs.io> (accesat în 14.9.2025).
- [11] Echipa Vite, *Ghidul Vite*, Documentația Vite, Documentație oficială Vite; citată pentru configurarea build-ului comună aplicațiilor și bibliotecilor și integrarea cu țintele Nx., 2025, URL: <https://vitejs.dev/guide/> (accesat în 14.9.2025).
- [12] Echipa Vitest, *Ghidul Vitest*, Documentația Vitest, Documentație oficială Vitest; citată pentru configurarea și utilizarea testelor unitare, folosită în proiect începând cu august 2025., 2025, URL: <https://vitest.dev/guide/> (accesat în 12.8.2025).
- [13] Google Firebase, *Autentificarea în Firebase*, Documentația Firebase, Documentație oficială Firebase; citată pentru autentificarea planificată și politicile de autorizare bazate pe roluri., 2025, URL: <https://firebase.google.com/docs/auth> (accesat în 10.7.2025).
- [14] Grupul Global de Dezvoltare PostgreSQL, *Documentația oficială PostgreSQL*, Documentația PostgreSQL, Documentație oficială PostgreSQL; citată pentru semantica SQL, suportul JSONB și siguranța tranzacțiilor utilizate prin Prisma., 2025, URL: <https://www.postgresql.org/docs/> (accesat în 18.10.2025).
- [15] Prisma Labs, *Documentația Prisma ORM*, Documentația Prisma, Documentație oficială Prisma; citată pentru modelarea schemei, migrații, generarea clientului TypeScript și integrarea cu Zod., 2025, URL: <https://www.prisma.io/docs> (accesat în 18.10.2025).
- [16] Proiectul tRPC, *Documentația tRPC — API-uri type-safe end-to-end*, Documentația tRPC, Documentație oficială tRPC; citată pentru contractele de tipuri între client și server și pentru structura router/context., 2025, URL: <https://trpc.io/docs> (accesat în 14.9.2025).
- [17] Redis, *Documentația Redis*, Documentația Redis, Documentație oficială Redis; citată în planul de dezvoltare pentru strategiile de caching și gestionare a sesiunilor de interogare., 2025, URL: <https://redis.io/docs/latest/> (accesat în 20.10.2025).
- [18] Vercel, *Vercel CLI — Command Line Interface*, Documentația Vercel, Documentație oficială pentru gestionarea proiectelor și a variabilelor de mediu în Vercel., 2025, URL: <https://vercel.com/docs/cli> (accesat în 16.12.2025).